Applying the agent paradigm to network management

R G Davison, J J Hardwicke and M D J Cox

Network management systems are large, distributed software systems that can suffer from the scalability and flexibility issues common in today's technology. Software agent technology offers the promise of a solution to these problems. A set of agent concepts have been adopted and used to construct an experimental system in the network management domain of ATM performance management. An evaluation of the system confirms that the software agents approach allows development of scalable network management systems that are cheaper to build, change and discard.

1. Introduction

The telecommunications business is in a new era of dynamic, highly competitive markets that continually place new and more demanding requirements on networks and services. These requirements are being matched by new networking technology and new service delivery mechanisms where the emphasis is on flexibility and scalability[1]. The network management area also needs to rise to the challenges of the future and to ensure it does not become the technology that prevents rapid response.

A set of characteristics have been identified that, it is believed, should be exhibited by future network management systems. Current network management architectures and systems already meet many requirements such as reliability and availability. However, in common with most of today's large software systems, they are difficult to modify to match changing requirements and can scale badly.

Software agent technology is one technology that offers useful concepts to address these concerns. The precise definition of the term 'software agent' is controversial so here a pragmatic approach is taken. An agent-based system is considered to be composed of many, loosely-coupled, distributed components that exhibit autonomy and that can work together through co-operation rather than centralised control.

Section 2 elaborates on the motivations for a new network management architecture, while section 3 outlines the properties of a new architecture based on software agent concepts. In section 4 a practical investigation of applying these concepts to one of the more challenging network management problems is described — the performance management of asynchronous transfer mode (ATM) networks. Section 5 considers a number of issues which are still to be addressed, while in section 6 conclusions are drawn to the effect that significant progress has been made with the agent approach but that further work is required as the development of a new architecture and supporting technology for network management is a challenging task.

[1] By flexibility we mean that inevitable changes in networks and services can be met as far as possible by run-time reconfiguration and by rapid deployment of smaller software components. By scalability we mean that the run-time performance does not degrade exponentially as both the network and the number of users expand.

2. Motivation for a new approach

Network management challenges the frontiers of software engineering due to its scale (for example 7500 exchanges and millions of multiplexers and line terminating equipment) and its complexity (many different equipment types, many different technologies, many different vendors, complex interconnection). The development of the Telecommunications Management Network (TMN) standard [1] in the late-1980s represented a significant breakthrough in network management. The standard specifies an architecture to structure management systems and defines interfaces and protocols to support software distribution. TMN is not the only standardised network management architecture but it shares key concepts with other approaches such as that of the Internet Engineering Task Force (IETF) [2].

The market-place in which the TMN concepts were developed was one characterised by a dominant, monopolistic public network operator (PNO) that co-operated with its peers to provide international services and networks. In such a market-place, the main focus was on an orderly, planned approach to the introduction of telecommunications management systems primarily intended for use by the PNO's operations and maintenance staff. A modest rate of change in telecommunications services and corresponding management activities was the goal. This

view is no longer valid as the market for telecommunications becomes more open and competitive. Network operators are now free to move around the globe with their service offerings and they now compete as well as co-operate with each other. They need to introduce new services quickly and require management systems that allow them to do so. They also have to drive investment and operational costs down. These changes lead to questions about the degree of fit between the original needs which drove TMN and the needs of today.

Three key challenges can be identified for current network management systems.

- Lengthy development life cycles

  Today's systems tend to comprise a small number of large software components. Large components hinder flexibility because of the long and expensive life cycles associated with them. Significant investment must be spent analysing, designing and implementing each component. If requirements change once the system is operating, the time and cost of modification and testing can become prohibitive. The level of investment in each component results in resistance to discarding it.

- Too much human involvement

  Network management still requires skilled (and expensive) human operators who may not be able to cope with the increasing complexity and dynamism of networks and services. Systems tend to be means for gathering and presenting information to operators. Architectures such as TMN support such an approach by implicitly encouraging workstation-size functions and encouraging software to be structured to match human organisation. This approach requires the communications infrastructure to pass the information. It requires software to receive, process and pass on information. It requires computing resource to process the information. The result is a heavy infrastructure that can be expensive, complicated and difficult to change.

- Architectures which do not scale

  TMN recognised that monolithic, centralised systems do not scale well and introduced distributed systems made out of a few large components each of which viewed part of the whole network. The size of these components can still lead to a level of scalability that will be too low in the future. The management system is typically organised into a hierarchy where lower levels monitor a smaller part of the network and then provide a hopefully abstract view to higher levels. In practice, the lack of good abstractions normally leads to higher levels in the hierarchy having to cope with a significant subset of the information seen by several of the lower level components.

3. Towards a software agent approach

The approach taken here is to replace systems with large, hierarchically structured, inter-dependent components with systems comprising isolated sub-systems each with a single layer of agents. The use of the term 'agent' here refers to an autonomous software component that exists in a community with other agents and interoperates with those agents without a predetermined control hierarchy.

Much of agent research is aimed at producing components with behaviour described as intelligent that can act autonomously, possibly representing a human being. Here, no claims are made for the intelligence of these agents — in fact simplicity is the goal when designing an agent. The interest here lies in building large-scale systems consisting of multiple heterogeneous and evolving applications. To this end, software agent technology promises adaptable and evolvable systems with inherent scalability [3]. Hence, it is the possible strengths of agent technology in inter-operability, loose component coupling and decentralised control that are being applied here rather than any more rarefied claim of intelligent behaviour.

The following are key characteristics in this approach.

- Decomposition into isolated sub-tasks

  The network management task to be addressed is decomposed into many sub-tasks, which can be performed in isolation. This permits the construction, or purchase, of a sub-system to perform each sub-task without dependencies on other sub-systems. It is worth noting that today's systems that use common functional decompositions, such as the FCAPS model from OSI [4], will not achieve this as the sub-tasks are not independent. The production of a set of independent sub-tasks for network management will be a major undertaking for which the experiences described in section 4 represents a useful start.

- Simple agents

  If a system comprises a large number of simple [2] agents, each one only requires a limited amount of program code devoted to solving its sub-task. This results in a smaller life cycle, making each agent faster and cheaper to create or modify. As a result there will be less cost in modifying agents and less reluctance to replace them when necessary.

- Autonomy

  Each agent in the system has responsibility to decide when to take action and what the action should be. There is no central controller deciding the system's

[2] Simplicity is a design goal rather than a well-defined characteristic.

operation since such a controller would not have the scalability characteristics required. The large number of agents make it unfeasible for each of them to incorporate any human processing. The system as a whole should require no human intervention for day-to-day operation. Rather the human role will now be one of strategic control — setting and modifying goals for the automated management system.

- No global data views

  Each sub-task is partitioned and responsibility for each partition assigned to an agent. The agent only maintains knowledge of its own partition and each partition is kept as small as is sensible. This approach is a key factor in ensuring scalability since as the network grows it requires more agents to be added rather than more data to be passed to existing components. The number of agents will grow, but the computational load and data collection requirements of each agent does not.

- No built-in control relationships

  On occasion, a problem may span more than one agent's view of the network. In this case, agents can exchange information and form a group for the duration of performing the task only. No permanent grouping is imposed upon the agents when they are designed. This results in a simple, single-layer system with less of an infrastructure overhead than a hierarchical system where information needs to be passed between layers. The absence of permanent groupings will also lead to a more flexible system that is better able to cope with change.

- Negotiation strategies

  If agents need to co-ordinate their actions then negotiation strategies are used as a design pattern and have proved especially useful for co-ordinating resource allocation [5].

4. Experimental virtual path management system

Using the ideas described in section 3, an experimental ATM performance management system has been built. The functionality of the system is summarised, then brief details are given of its design and implementation, followed by an example illustrating the system in operation.

4.1 The problem domain

ATM technology offers the potential for creating a single network that can carry a highly diverse range of services. This makes the task of managing an ATM network greater than for current networks. Functions concerned with ensuring efficient delivery of contracted quality of service for each service type will be challenging and were deemed suitably demanding for these experiments into new network management systems. The work described in this paper is based upon a deployment scenario of ATM to support a large, public ATM network carrying a range of services including both data and delay-sensitive services. The network will have call control using signalling and its own protocols for traffic and congestion control. Further details on the ATM performance management domain can be found in Davison and Azmoodeh [6].

The goal is to build a system that avoids unnecessary connection rejection when there is spare capacity in the network. This occurs when actual traffic volume differs significantly and unevenly from that used to dimension the network. This could be due to a change in service popularity, perhaps because of a marketing campaign or tariff change, or due to the introduction of a completely new service.

When this occurs, parts of the network become congested while other parts may be under-utilised, resulting in connections being rejected that could be accepted if the traffic load were better balanced. An ATM virtual path connection (VPC) is a semi-permanent connection that is used to transport a large number of simultaneous connections across one or more links. A connection will be rejected from the network when the capacity reserved for the VPC it is to traverse has been used by existing connections to the point where there is not enough left for the new connection. A VPC in this state is referred to as 'congested'.

The management task is to minimise the occurrence of congested VPCs. This can be decomposed into two independent sub-tasks:

- recognise a congested VPC exists that shares part or all of its route with VPCs that are under-utilised, and re-distribute spare capacity from those VPCs to the congested VPC — this task is called VPC bandwidth tuning,

- recognise a congested VPC exists that does not share part of its route with VPCs that are under-utilised, re-route a VPC away from the congested links — this task is called VPC topology tuning.

Topology tuning will in itself not overcome a congested VPC but it will create spare link capacity for bandwidth tuning to operate effectively and thus overcome the congestion.

4.2 Design and implementation

The tasks of bandwidth tuning and topology tuning were implemented as separate agent communities (Fig 1).

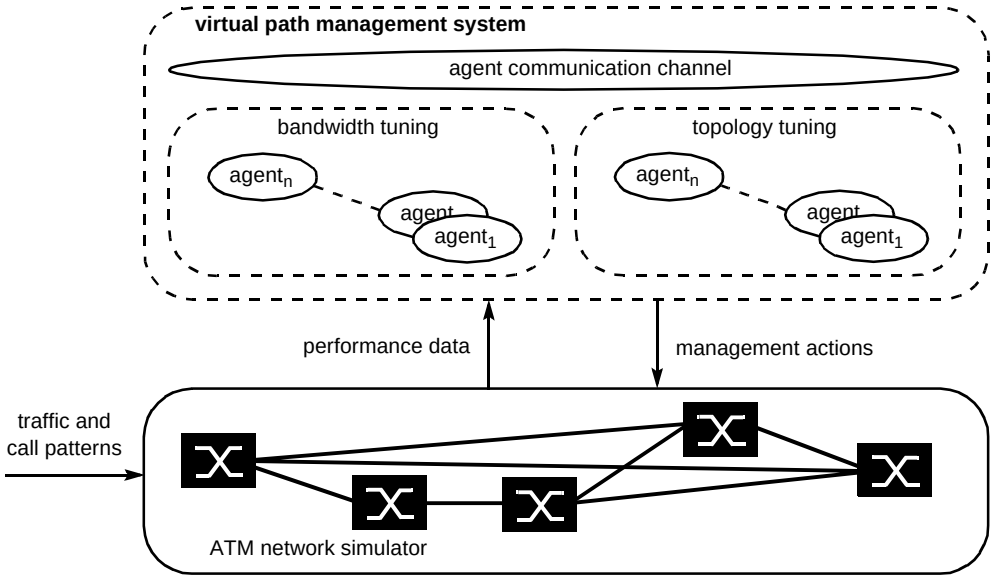

Fig 1 Organisation of the virtual path management experiment.

For bandwidth tuning, the network was partitioned so that each agent was responsible for managing VPCs on a single link. Congestion can be solved without agent interaction if the congested VPC spans only the local link and can be given bandwidth from other VPCs which also span only the local link. If one of the VPCs in the solution spans more than one link, then the agents must negotiate and exchange information to decide who chooses how much bandwidth to move.

In contrast, for topology tuning the network was partitioned into two sets of agents — one set monitoring links and the other managing VPCs. The link agent's role is to recognise when a link is congested and inform the VPC agents monitoring that link of the problem. The VPC agents then negotiate to decide the most appropriate VPC to move.

The agents were built using an in-house, experimental agent platform that provided developers with support to implement each agent as a separate lightweight process and an inter-agent communications channel [7] providing an asynchronous message passing service.

An *ad hoc* agent communication language was specified which has strong similarities to the recently published FIPA ACL [7, 8]. The whole system was implemented in Lisp. A burst-scale ATM network simulator was developed that allows simulation of networks in close to real time. The simulator provides a management interface that permits data to be extracted and management actions to be applied during simulation.

4.3 An example operation

The operation of the two agent communities is best understood through an example. Figure 2 illustrates a network fragment showing four VPCs carried across two

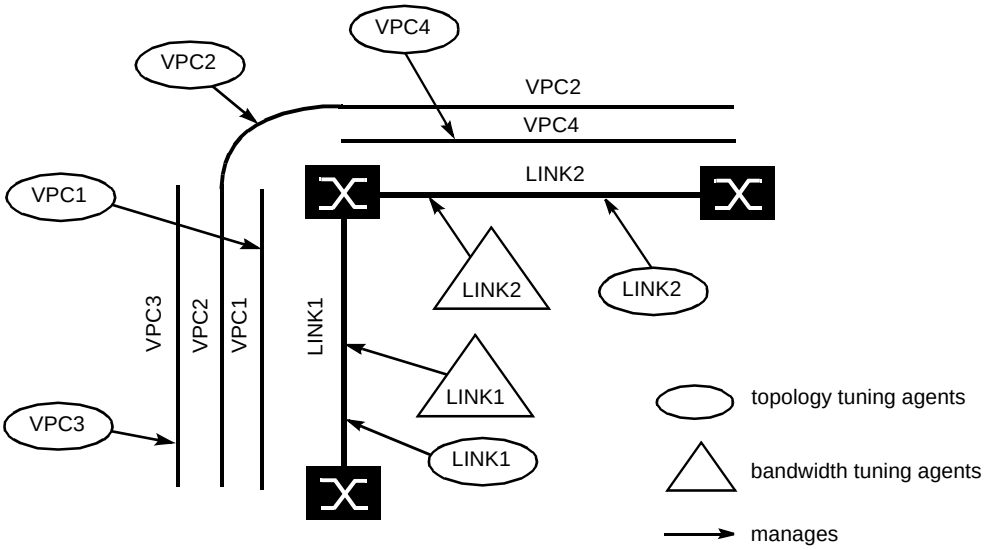

Fig 2 Example network fragment.

physical links. The bandwidth and topology tuning agents are also shown.

An example congestion scenario will now be described which requires the operation of both bandwidth and topology tuning agents. The messages sent between agents during the scenario are shown in Fig 3.

At the start of the scenario (T1 in Fig 3), VPC2 is congested. The bandwidth tuning agents for LINK1 and LINK2 which carry VPC2 independently recognise this and negotiate to decide which of them manages the busiest link that will place the tightest constraint on increasing the bandwidth available to VPC2. This is the agent managing LINK2. This agent then informs the other agents how much extra bandwidth to reserve for the congested VPC2. The agents then all locally redistribute their spare capacity as appropriate.

Sometime later in the scenario (T2 in Fig 3) VPC1, VPC2 and VPC3 all become congested. The bandwidth

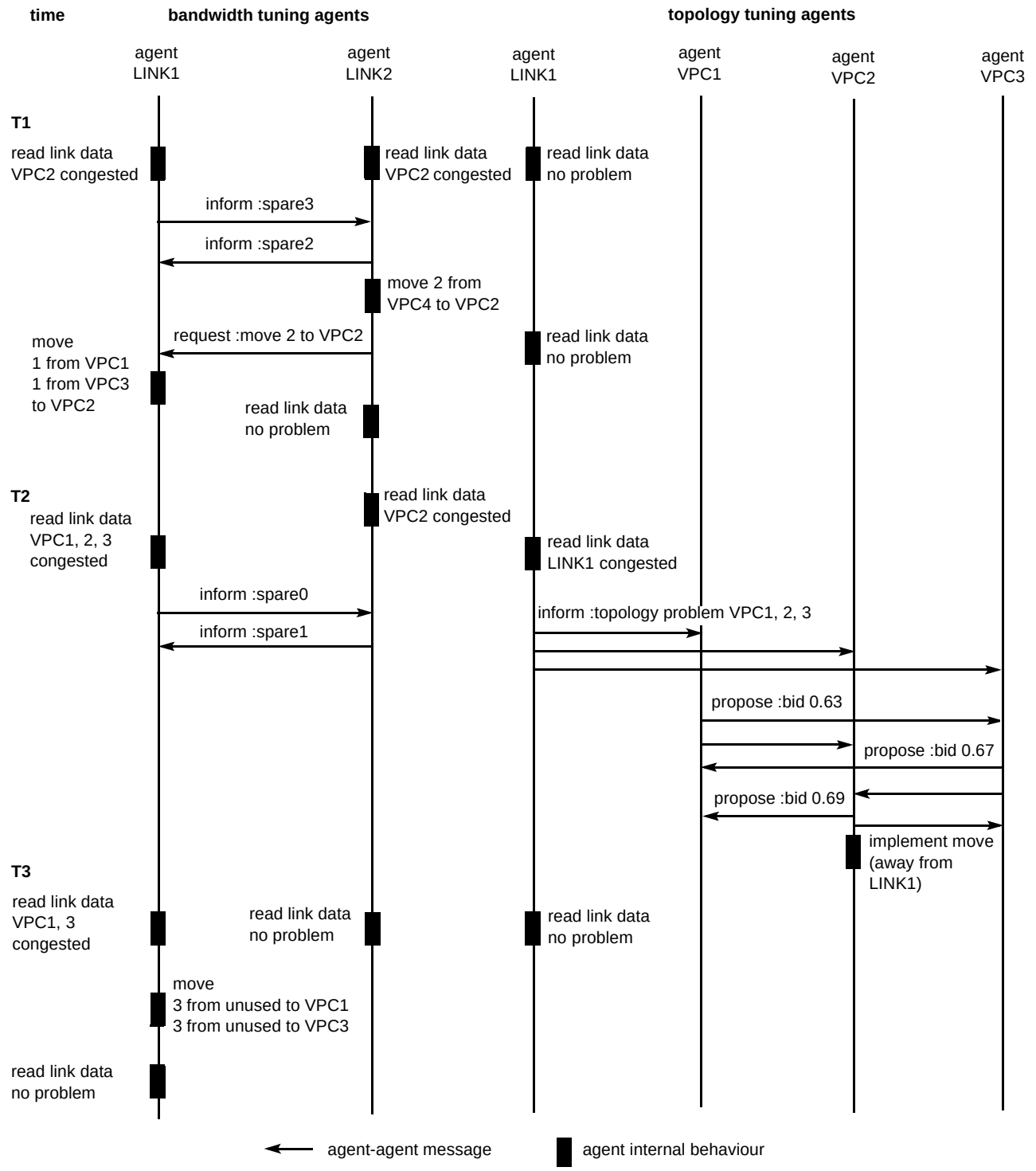

Fig 3 Message sequence diagram for the performance management scenario described in section 4.3.

tuning agents are unable to solve this problem by re-distributing spare capacity because LINK1 has no spare capacity.

The topology tuning agent monitoring LINK1 recognises this situation from its network monitoring and informs the topology agents for VPC1, VPC2 and VPC3 of the problem. The VPC agents all propose a 'bid' to re-route off the congested link. The winning bid will be from the agent whose proposed move offers the best balance between bandwidth moved and the increased potential for congestion along the new route. VPC2 agent wins and re-routes its VPC away from LINK1.

Then (T3 in Fig 3) the bandwidth that was reserved for VPC2's use becomes available for re-distribution and is used by the bandwidth tuning agent for LINK1 to solve the congestion on VPC1 and VPC3.

A key point from this scenario is that there is no direct communication between the bandwidth and topology agent communities. The two sub-systems are able to operate independently even though they control the same resources. This reflects the low degree of coupling being aimed for and leads to less dependencies between software and hence systems that are easier to change.

4.4 Experimental results

The system did achieve its functional goal of avoiding congestion in various traffic scenarios. For example the graphs in Figs 4 and 5 together illustrate a typical congestion scenario.

Figure 4 shows the link utilisation for three links (the simulated network had more links, but only those relevant to this example are shown). An increase in traffic is simulated on the link with highest utilisation after 1200 seconds. When unmanaged, this link becomes congested, but there is spare capacity in the other two links illustrated. When managed, the topology tuning agents recognise the link becoming congested and take action by moving a VPC to a new route which uses the two links which have spare capacity. The graph shows the utilisation of the congested link reducing as it is relieved of the traffic on the moved VPC. Likewise the other links' utilisation rises to accommodate the traffic of this VPC.

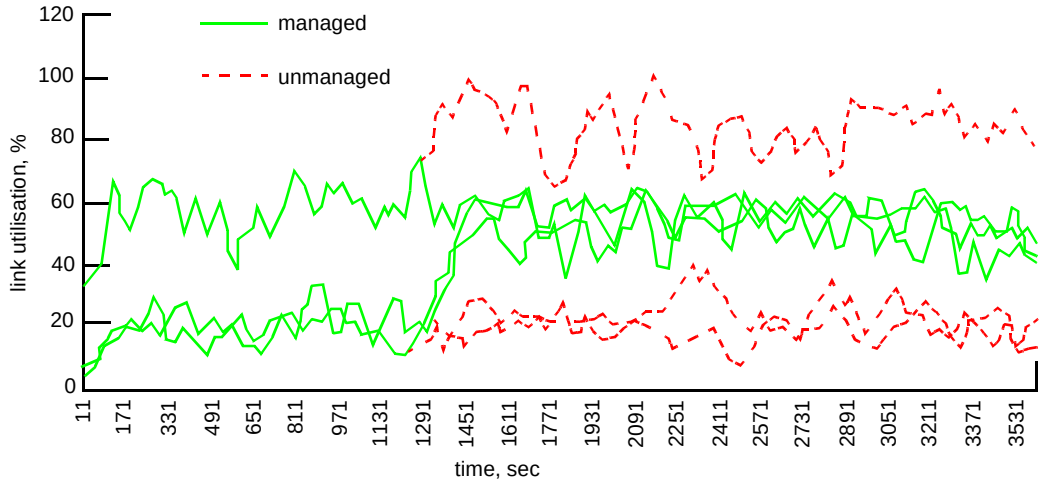

Fig 4 Link utilisation with and without management agents.

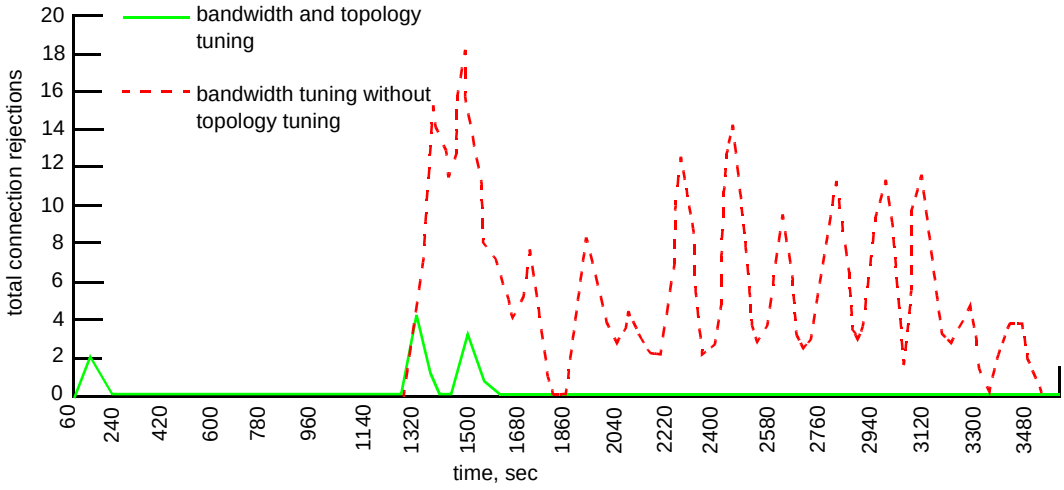

Fig 5 Connection rejection with and without topology tuning.

Figure 5 shows the numbers of connections rejected on the links illustrated in Fig 4. It shows how the actions taken to balance load across the links by re-routeing a VPC, result in increased revenue as fewer connections are rejected.

5. Issues to be addressed

Although the experiments illustrate strong results and reinforce belief in the approach as a way forward, there are several issues that need to be addressed before the approach could be recommended for real deployment.

Development of distributed systems with such highly decentralised algorithms will depend on the availability of design methods and development tools that allow architects, designers and programmers to reason, understand and debug it. Experience has shown that, without this support, unfavourable interactions which can lead to deadlock, loops and inappropriate actions are hard to discover and overcome. Additionally, further support is needed when designing decentralised algorithms to ensure that the number of messages needed to solve a problem does not increase exponentially with the size of the problem domain.

Real deployment of such systems will need distributed computing platforms that can cope with high levels of small grained distribution, much higher than is prevalent in distributed systems today (e.g. those built with CORBA). The requirements for naming and addressing of agents needs to be developed.

The software configuration of the system becomes more complex because the number of components in the system has increased. There may be a trade-off point at which the granularity of agents becomes too small and the resulting scale and flexibility benefits are outweighed by configuration management costs. The upgrading and addition of the agents that comprise the system must be performed in a totally automated manner to cope with the potential volume of work. A number of emerging technologies currently show promise for solving this including the use of ‘push’ technologies [9] which automates the distribution of software updates, and mobile code [10] which can be used to distribute code as needed for a particular problem rather than install it everywhere in case it is needed.

The experimental system described in this paper used an *ad hoc* inter-agent communication language. Bodies such as FIPA are working to standardise such languages. If they succeed, this could result in the availability of better agent development tools.

With many different groups of agents all operating on the same network to cover an operator’s management needs, the translation of that operator’s business policies into a consistent set of goals for each agent in the system is an area still to be addressed.

6. Related work

Much of the traditional approaches to VPC management treat the problem as one of constrained optimisation. These apply specialised nonlinear optimisation techniques. There are many papers of this type including Logothetis et al [11] and Cheng and Lin [12]. These approaches differ from those described here in that they generally imply a centralised algorithm responsible for the whole network and overstress the importance of optimality.

An example of a TMN-based solution to VPC management can be seen in the work of the RACE ICM project [13]. The VPC topology management and VPC bandwidth management are treated as separate but centralised functions.

Like the approach here, the work of Somers [14] advocates an agent approach. However, that work has more of a focus on a generic internal agent architecture and favours a hierarchical organisation of agents based on geography.

Hayzelden and Bigham [15] also choose an agent approach but one organised in layers. Although their lower level agents are isolated, their VPC topology and VPC bandwidth concerns are treated together within the higher layer.

7. Conclusions

The use of software agent technology has been proposed as a promising candidate for building future network management systems. It offers concepts that can result in more scalable and flexible systems that are cheaper to build, change and discard. An experimental system for managing ATM virtual paths was built to evaluate these claims.

There are still many issues to be addressed before the proposed approach becomes industrially viable. However, this initial work has shown the significant contribution that it could make and it is believed that the concepts described in this paper will form part of a much-needed, new generation of network management systems.

References

1 ITU-T Recommendation M.3010: ‘Principles for a telecommunications management network’, (May 1996).

2 IETF RFC1157: ‘A simple network management protocol’, ftp://ftp.isi.edu/in-notes/rfc1157.txt

3 Finin T: ‘Knowledge sharing KQML, KIF and Ontologies’, tutorial, Practical Application of Intelligent Agents and Multi-Agent Technology, London (April 1997).

4 ITU-T Recommendation M.3400: ‘TMN management functions’, (April 1997)

5 Nwana H S, Lee L and Jennings N R 'Co-ordination in software agent systems', BT Technol J, 14, No 4, pp 79—88 (October 1996).

6 Davison R G and Azmoodeh M: 'Performance management of public ATM networks', Proceedings of the 5th Symposium on Integrated Network Management, California (May 1997).

7 Foundation for Intelligent Physical Agents (FIPA), http://www.cselt.stet.it/fipa/

8 O'Brien P D and Nicol R C: 'FIPA — towards a standard for software agents', BT Technol J, 16, 3, pp 51—59 (July 1998).

9 Enabling Self-Managing Applications, http://www.marimba.com/datasheets/castanet-ds.html

10 Baldi M, Gai S and Picco G 'Exploiting code mobility in decentralised and flexible network management', Proceedings of the first Mobile Agents Workshop, Massachusetts (April 1997).

11 Logothetis M, Shioda S and Kokkinakis G: 'Optimal virtual bandwidth management assuring network reliability', Proceedings IEEE Conference on Communications, pp 30—36 (1993).

12 Cheng K and Lin F: 'On the joint virtual path assignment and virtual circuit routing problem in ATM networks', Proceedings IEEE Globecom '94, 2, pp 777—782 (1994).

13 RACE: 'Updated TMN Architecture and Functions', ICM Deliverable 16 (December 1994).

14 Somers F: 'HYBRID: intelligent agents for distributed ATM network management', in IATA'96 Workshop at ECAI'96, Budapest, Hungary (1996).

15 Hayzelden A and Bigham J: 'Hetrogeneous multi-agent architecture for ATM virtual path network resource configuration', to appear in IATA'98 workshop held at Agents World '98, Paris, France (1998).

Rob Davison graduated from Bristol University with a degree in Computer Systems Engineering in 1998. He joined BT's Speech and Language division and spent 2 years researching the application of AI techniques to spoken dialogue systems, before transferring into the network management research area. Since then he has worked on a range of research and development projects covering areas such as fault diagnosis, service provisioning, broadband performance management and distributed computing. He currently leads the network management research group with interests in new architectures for network management, open network control and the management of future ATM and IP networks.

Jim Hardwicke joined BT in 1981 as an apprentice working in London and subsequently graduated with a first class BSc honours degree in Computer Science from Loughborough University of Technology in 1991. He moved to BT Laboratories in the same year and worked as a software engineer on a number of BT's large network management projects. He joined the network management research group in 1996 where he has worked on the application of new software technology to network management.

Michael Cox graduated from Brunel University in 1987 with a BSc in Physics and Computer Science. He joined BT Laboratories in Ipswich working in the transmission domain surveillance (TDS) programme. He was primarily responsible for the design and implementation of a prototype PDH fault correlation expert system through several live field trials and then integrating it into the TDS fault manager product. In 1995 he moved to the network management research team in Applied Research and Technologies where he has been experimenting with ideas from agent and constraint satisfaction technologies for the performance management of ATM networks. In 1997 he obtained an MSc in Computer Science (Artificial Intelligence) at the University of Essex.